Assignment: Data Preprocessing & Exploratory Data Analysis

Course: Data Analytics Lab

Department of Computer Science and Engineering

General Instructions

1. **Environment:** Work inside a Jupyter Notebook or Google Colab notebook.
2. **Reproducibility:** Set `random_state = 42` wherever applicable.
3. **Submission:** Submit your executed `.ipynb` file along with an exported PDF showing all outputs and generated visualizations.
4. **Dataset:** Use the `diamonds` dataset available natively via Seaborn.

Part 1: Data Preprocessing

Load the dataset using the following code block:

```
import numpy as np
import pandas as pd
import seaborn as sns

# Load dataset
df = sns.load_dataset('diamonds')

# Inject missing values for the assignment
np.random.seed(42)
mask_carat = np.random.rand(len(df)) < 0.05
mask_price = np.random.rand(len(df)) < 0.05

df.loc[mask_carat, 'carat'] = np.nan
df.loc[mask_price, 'price'] = np.nan
```

Task 1: Dataset Inspection & Invalid Value Cleaning

1. Display the shape, data types, and non-null counts using `.info()` and summary statistics using `.describe()`.
2. Identify missing value counts and missing percentages for all columns.
3. Inspect the physical dimension columns `x`, `y`, and `z`. A diamond cannot have a physical dimension of 0 mm. Identify how many rows have `x == 0`, `y == 0`, or `z == 0`, replace these zeros with `NaN`, and report the updated missing count.

Task 2: Missing Value Imputation

1. For numeric feature `carat`, impute missing values using its **median**.
2. For numeric target `price`, impute missing values using its **median**.
3. For dimension columns `x`, `y`, and `z` (where 0 mm entries were converted to `NaN`), impute missing values using the **median of their respective columns grouped by** `cut`.
4. Confirm that there are 0 missing values left across the entire DataFrame.

Task 3: Categorical Feature Encoding

1. Inspect the unique categories of `cut`, `color`, and `clarity`.
2. `cut` and `clarity` are ordinal categorical variables. Encode them using explicit integer mappings that preserve their natural quality ordering:
   - `cut` **mapping:** `'Fair': 0, 'Good': 1, 'Very Good': 2, 'Premium': 3, 'Ideal': 4`
   - `clarity` **mapping:** `'I1': 0, 'SI2': 1, 'SI1': 2, 'VS2': 3, 'VS1': 4, 'VVS2': 5, 'VVS1': 6, 'IF': 7`
3. `color` is a nominal categorical variable. Convert it into numeric format using One-Hot Encoding (`pd.get_dummies()`) with `dtype=int`. Display the first 5 rows of the updated DataFrame.

Task 4: Outlier Detection & Capping (Winsorization)

1. Calculate $Q1$, $Q3$, and IQR for the `table` column. Compute the upper and lower boundary thresholds using:

$$\text{Lower Bound} = Q1 - 1.5 \times IQR$$

$$\text{Upper Bound} = Q3 + 1.5 \times IQR$$

2. Count how many outliers exist in `table`.
3. Treat these outliers by capping (clipping) the values to the lower and upper bounds so no rows are deleted.
4. Plot box plots of `table` **before** and **after** capping side-by-side to verify the transformation.

Task 5: Train/Test Split & Data Scaling

1. Separate your dataset into features X (all predictors) and target variable y (`price`).
2. Perform an **80/20 train/test split** using `train_test_split()` with `random_state = 42`. Print the shapes of `X_train`, `X_test`, `y_train`, and `y_test`.
3. Apply **Standardization (Z-score scaling)** to continuous features (`carat`, `depth`, `table`, `x`, `y`, `z`).
   - **Requirement:** Fit the `StandardScaler` **only** on `X_train` and then transform both `X_train` and `X_test`.
   - Briefly explain in 1–2 sentences why fitting the scaler on the full dataset before splitting causes data leakage.

Part 2: Exploratory Data Analysis (EDA)

Task 6: Univariate Analysis & Target Transformation

1. Plot a histogram with 30 bins for `price` in the training set and describe its skewness (left-skewed, symmetric, or right-skewed).
2. Plot a histogram for `carat`.
3. Create a count plot showing the distribution of diamond `cut` categories.
4. Apply a Log Transformation ($\ln(1+\text{price})$) to create `log_price`. Plot its histogram alongside the original `price` distribution and comment on how the shape changes.

Task 7: Bivariate Analysis

1. Compute the average diamond `price` grouped by `cut` using `.groupby()` and print the result.
2. Create a box plot comparing `price` across different `cut` levels.
3. Create a scatter plot of `carat` (x-axis) vs `price` (y-axis). Observe whether the relationship is strictly linear or curved.

Task 8: Correlation Analysis & Heatmap

1. Compute the correlation matrix for all numeric columns in `X_train` along with `price` using `.corr()`.
2. Generate an annotated `seaborn` heatmap using the `coolwarm` palette.
3. Identify which feature has the strongest positive correlation with `price`.